Sistemas Distribuidos

Jesus Martínez Trejo

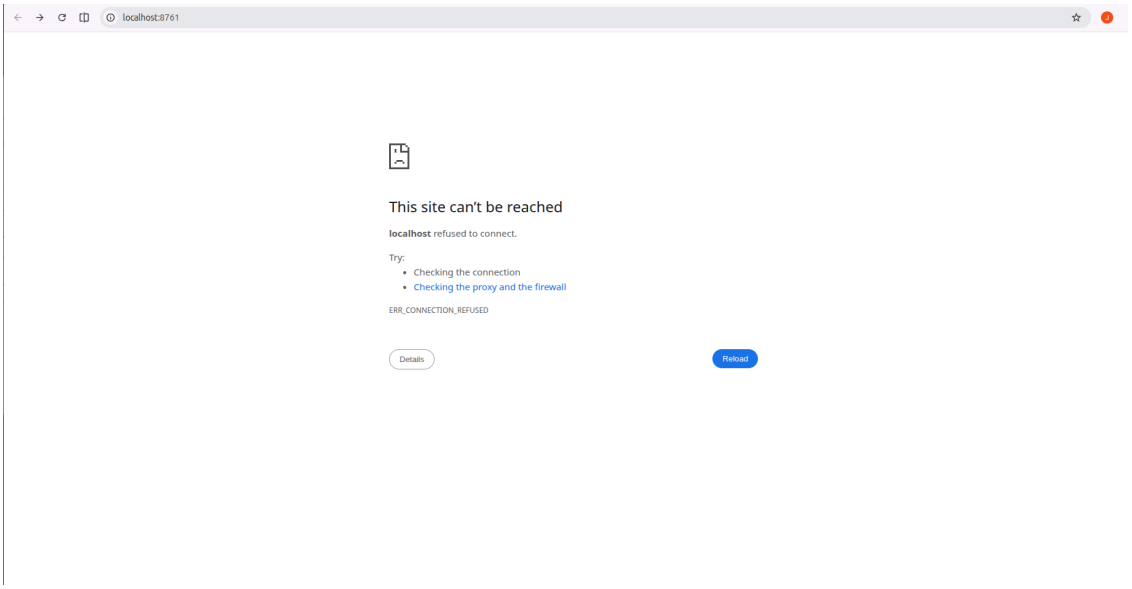
Mexi Banco: de un monolito a servicios que se encuentran solos

1. Cambio de arquitectura

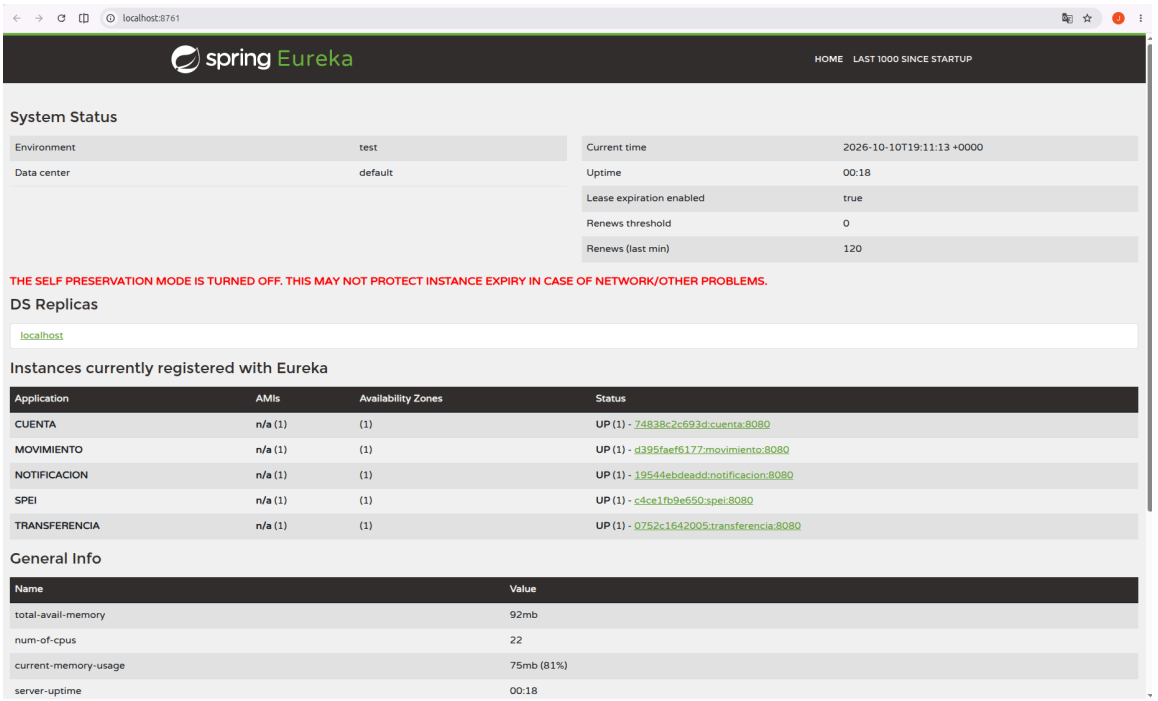
En esta práctica pasamos de tener un monolito con una sola base de datos encargada de las operaciones del negocio simulado, donde una transferencia era simplemente una llamada a un método. En ese modelo, si algo fallaba, la base de datos transaccional revertía todo automáticamente al confirmar todo o revertir toda la operación.
Al separarlo en microservicios que se comunican por peticiones HTTP, esta atomicidad se rompió. Cada actividad del negocio tiene su propia base de datos, por lo que, si una petición remota falla, los cambios previos ya no se deshacen por sí solos. Además, esta separación obliga a implementar una única puerta de entrada (gateway) para proteger las operaciones internas, junto a un directorio (discovery) que permite hacer el sistema tolerante a fallos al gestionar múltiples instancias de un mismo servicio y conocer de forma dinámica su ubicación lógica en la red.

2. Directorio (Discovery)

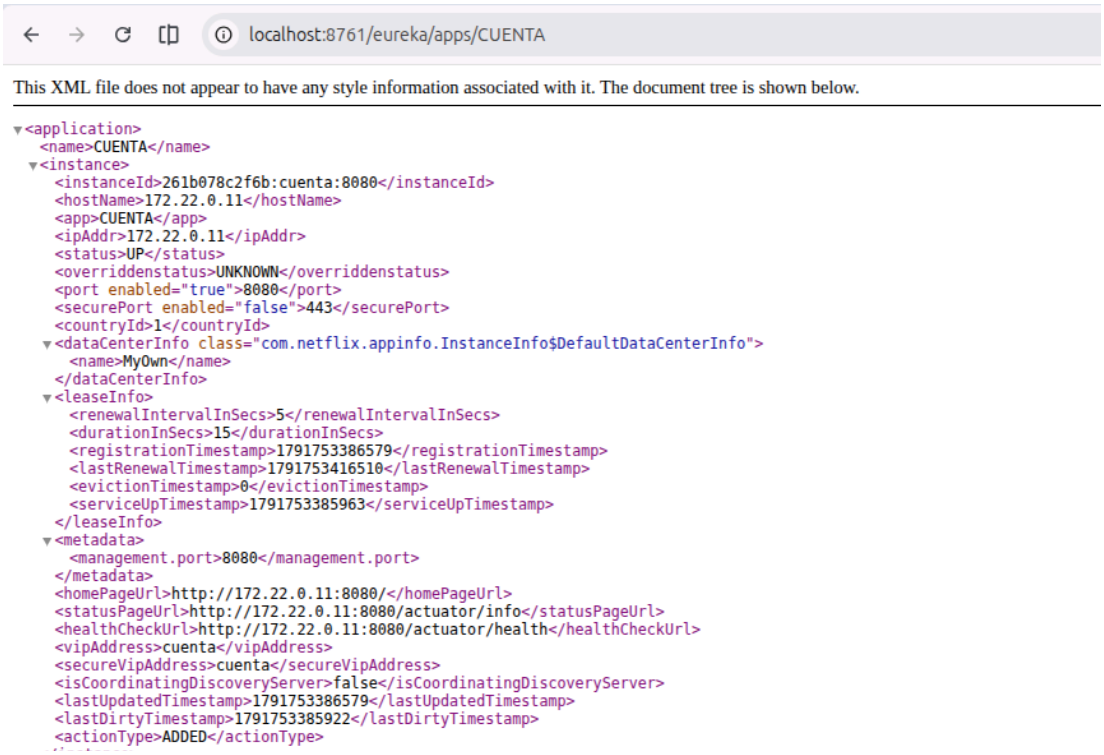
Antes de encender el discovery.



Después de encender Discovery



XML de cuenta



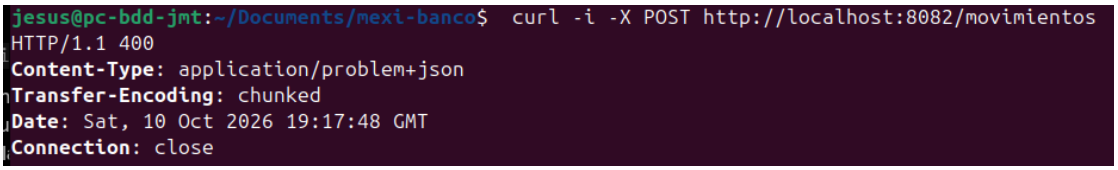
¿Alguien está utilizando el directorio?

Hasta este punto del laboratorio, ningún servicio estaba utilizando el directorio para encontrar a CUENTA. Aunque los cinco microservicios ya estaban registrados en Eureka enviando heartbeats, la comunicación interna dependía, en el caso de TRANSFERENCIA, de CUENTA_URL: http://cuenta:8080 escrita en el docker compose. Por lo que hasta el siguiente paso, encender el gateway, nadie estaba utilizando este directorio.

3. Puerta única de acceso (Gateway)

Error 400 en el puerto 8082

Al apuntar al puerto 8082, el endpoint interno responde directamente con un error de sintaxis, demostrando que la operación interna no está protegida contra llamadas externas.



Errores 404 por el gateway

El primer 404 lo produjo el servicio cuenta. El gateway dejó pasar la petición y la respuesta incluyó la cabecera X-Instancia confirmando que el microservicio procesó la solicitud pero no encontró la cuenta. El segundo 404 lo produjo el gateway. La petición se bloqueó en la puerta porque no hay rutas que permitan un POST a movimientos, lo cual se comprueba porque la respuesta no trae la cabecera X-Instancia.



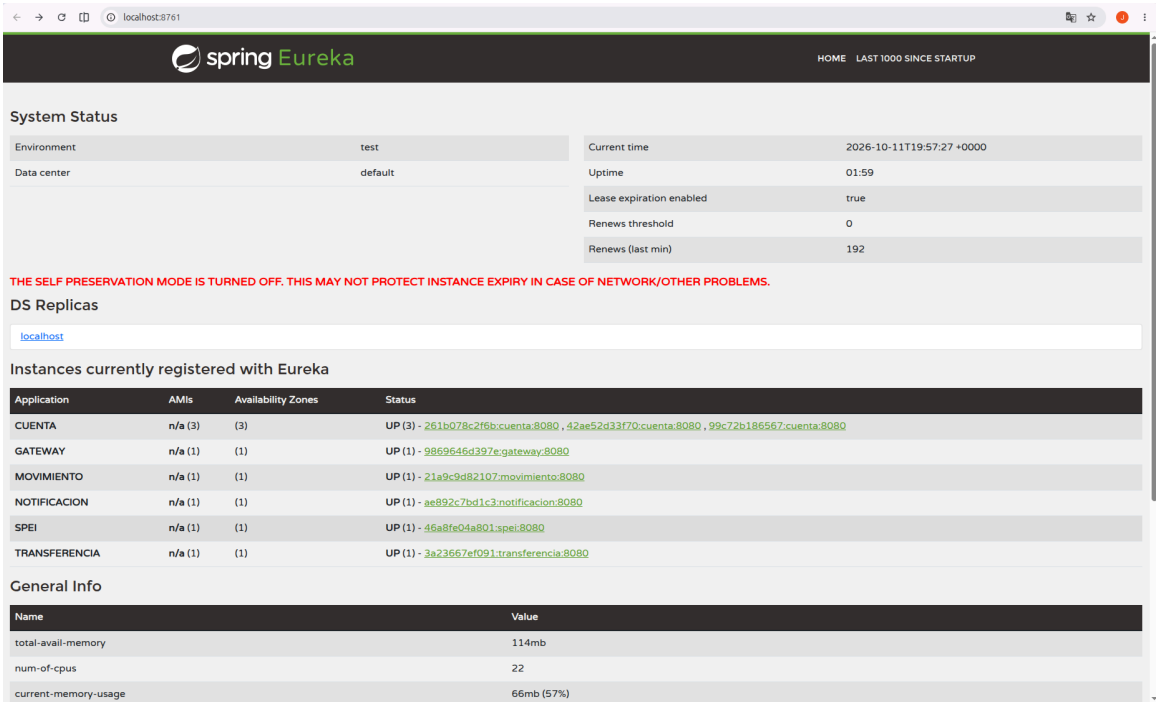
Creación de una cuenta (201) y consulta de la cuenta creada (200)



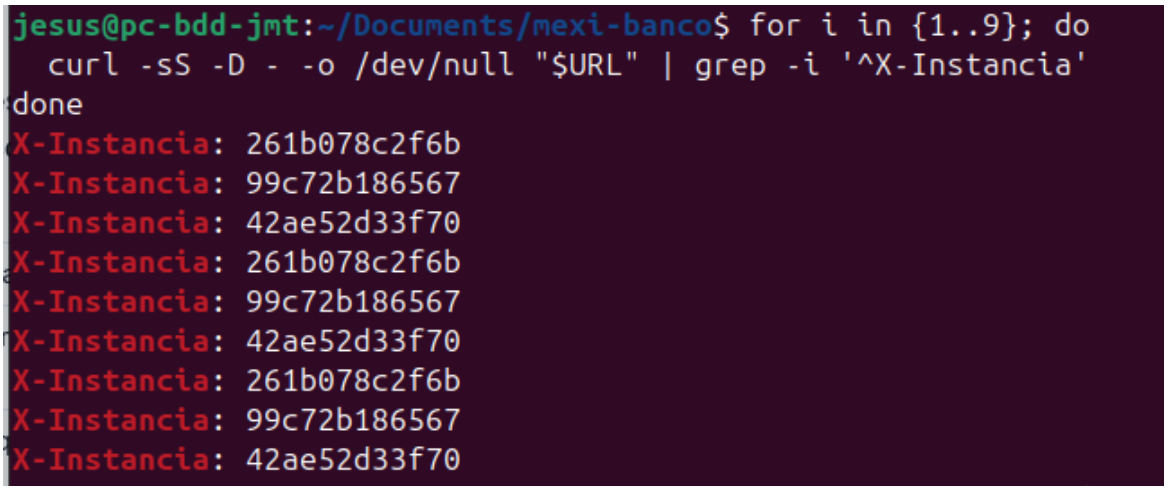
4. Réplicas de un servicio y fallos

Hora: 14:05 hrs.

Tablero de Eureka mostrando CUENTA UP (3)



Balanceo Round Robin a través de 9 consultas



Simulación de la caída del servicio

- Predicción: Antes de apagar la copia, supuse que fallarían solo algunas consultas dado el tiempo que tardaría en desconectar la referencia al servicio. Pensé que los errores iban a durar alrededor de 17 segundos entre el último heartbeat del servicio y la eliminación del

directorio, y que dejarían de fallar un poco después de que la copia desapareciera del directorio.

- Línea del tiempo
  El contenedor se apagó a las 14:07:08. Se observaron errores HTTP 500 intermitentes durante un periodo de aproximadamente 37 segundos (hasta las 14:07:45), contabilizando un total de 7 fallos alternados con respuestas exitosas (200 OK). Esto se debe a la propagación por capas de la baja del servicio. En primer lugar, el directorio Eureka esperó un periodo de tolerancia de 15 segundos sin recibir heartbeats para quitar a la instancia de su lista. En segundo lugar, dado que el Gateway no consulta a Eureka en tiempo real por cada petición, su caché local tardó cerca de 5 segundos en actualizarse. Durante este intervalo de desincronización, el balanceador del Gateway continuó enviando tráfico hacia la instancia fuera de servicio, lo que generó los errores intermitentes registrados.

```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
14:06:55 200
14:06:56 200
14:06:57 200
14:06:58 200
14:06:59 200
14:07:00 200
14:07:01 200
14:07:02 200
14:07:03 200
14:07:04 200
14:07:05 200
14:07:06 200
14:07:07 200
14:07:08 200
14:07:09 200
14:07:10 200
14:07:11 200
14:07:12 500
14:07:15 200
14:07:16 500
14:07:19 200
14:07:20 200
14:07:21 200
14:07:22 500
14:07:25 200
14:07:26 500
14:07:29 200
14:07:30 200
14:07:31 200
14:07:33 500
14:07:36 200
14:07:37 200
14:07:38 500
14:07:41 200
14:07:42 500
14:07:45 200
14:07:46 200
14:07:47 200
```

```
jesus@pc-bdd-jmt:~$ date +%T
docker kill mexi-banco-v06-cuenta-1
14:07:08
mexi-banco-v06-cuenta-1
```

5. Reflexiones

- El directorio resuelve saber qué servicios hay y dónde están, sin escribir direcciones a mano.
- El gateway resuelve tener una sola puerta de acceso y definir reglas de qué se puede llamar desde afuera.
- El balanceador resuelve repartir el trabajo entre varias copias y dejar de mandar tráfico a las que mueren.
- Cuando murió una copia vi errores intermitentes porque el gateway seguía usando una caché desactualizada mientras el directorio esperaba que venciera el plazo de desconexión.

6. Tarea

Ejecución del demo-v06.sh con los saldos finales

```
=== 5. Saldos: el dinero no se crea ni se pierde ===
$ curl http://localhost:8080/api/cuentas/002180009175053801
{
  "id": 2,
  "clabe": "002180009175053801",
  "titular": "Ana",
  "saldo": 800.00
}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009175053802
{
  "id": 3,
  "clabe": "002180009175053802",
  "titular": "Beto",
  "saldo": 700.00
}
[HTTP 200]
-> Ana 800.00 + Beto 700.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

=== 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) ===
$ curl http://localhost:8080/api/movimientos?clabe=002180009175053801
[
  {
    "id": 3,
    "clabeCuenta": "002180009175053801",
    "tipo": "TRANSFERENCIA_ENVIADA",
    "monto": -200.00,
    "saldoResultante": 800.00,
    "fecha": "2026-10-11T20:29:00.660296Z",
    "referencia": "a 002180009175053802"
  },
  {
    "id": 1,
    "clabeCuenta": "002180009175053801",
    "tipo": "DEPOSITO",
    "monto": 1000.00,
    "saldoResultante": 1000.00,
    "fecha": "2026-10-11T20:28:59.179237Z",
    "referencia": "saldo inicial"
  }
]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clabe=002180009175053802
[
  {
    "id": 2,
    "clabeCuenta": "002180009175053802",
    "mensaje": "TRANSFERENCIA_RECIBIDA por 200, saldo resultante 700.00",
    "fecha": "2026-10-11T20:29:00.841732Z"
  }
```

```
=== 7. SPEI de 50 con Idempotency-Key, dos veces (POST /api/spei) ===
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91750538' -d '{"claveOrigen":"002180009175053801","bancoDestino":"BANCO
-X","claveDestino":"012180000000000009","monto":50}'
{
  "id": 1,
  "idempotencyKey": "spei-demo-91750538",
  "claveOrigen": "002180009175053801",
  "bancoDestino": "BANCO-X",
  "claveDestino": "012180000000000009",
  "monto": 50,
  "estado": "ENVIADO"
}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-Key: spei-demo-91750538' -d '{"claveOrigen":"002180009175053801","bancoDestino":"BANCO
-X","claveDestino":"012180000000000009","monto":50}'
{
  "id": 1,
  "idempotencyKey": "spei-demo-91750538",
  "claveOrigen": "002180009175053801",
  "bancoDestino": "BANCO-X",
  "claveDestino": "012180000000000009",
  "monto": 50.00,
  "estado": "ENVIADO"
}
[HTTP 201]
-> mismo id (1 -> 1): no se cobro dos veces. Saldo de Ana: 750.00 (esperado 750)
```

```
=== 11. Los saldos no se movieron con los errores ===
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caida: ./demo-v06.sh --latido
```

Análisis de los saldos

A pesar de que el proceso reparte las tareas entre distintos contenedores, los saldos de las cuentas se mantuvieron consistentes. Ana inició con un saldo de 1000, envió una transferencia de 200 a Beto, y posteriormente realizó un SPEI de 50 hacia otro banco. El saldo resultante de Ana fue de 750, y el de Beto quedó en 700. El dinero no se creó ni se perdió durante las transacciones distribuidas.

Balanceo interno en Transferencia:

El log de ejecución (docker compose logs cuenta | grep -E 'cargos|abonos') demuestra que el servicio de transferencia dejó de utilizar el DNS estático de Docker. Gracias a @LoadBalanced, consultó dinámicamente el directorio Eureka y distribuyó la carga internamente. Esto se comprueba al observar que el cargo por la transferencia a la cuenta de Ana fue atendido por la instancia cuenta-3, mientras que el abono a la cuenta de Beto fue procesado por la instancia cuenta-1.

```
jesus@pc-bdd-jmt:~/Documents/mexi-banco$ docker compose logs cuenta | grep -E 'cargos|abonos'
cuenta-3  | 2026-10-11T20:29:00.573Z  INFO 1 --- [cuenta] [nio-8080-exec-5] mx.mexibanco.cuenta.CabeceraInstancia    : POST /cuentas/002180009175053801/cargos atendida por 261b078c2f6
cuenta-3  | 2026-10-11T20:29:03.416Z  INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia    : POST /cuentas/002180009175053801/cargos atendida por 261b078c2f6
cuenta-2  | 2026-10-11T20:29:02.453Z  INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia    : POST /cuentas/002180009175053801/cargos atendida por 42ae52d33f7
cuenta-1  | 2026-10-11T20:29:00.591Z  INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia    : POST /cuentas/002180009175053802/abonos atendida por 99c72b1865e
```